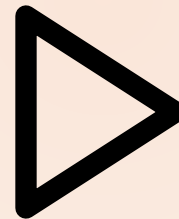


MODULE 04 · 30 MIN

Build Faster with Best-of-N

The first answer is rarely the best. Generate three; score; ship the winner.



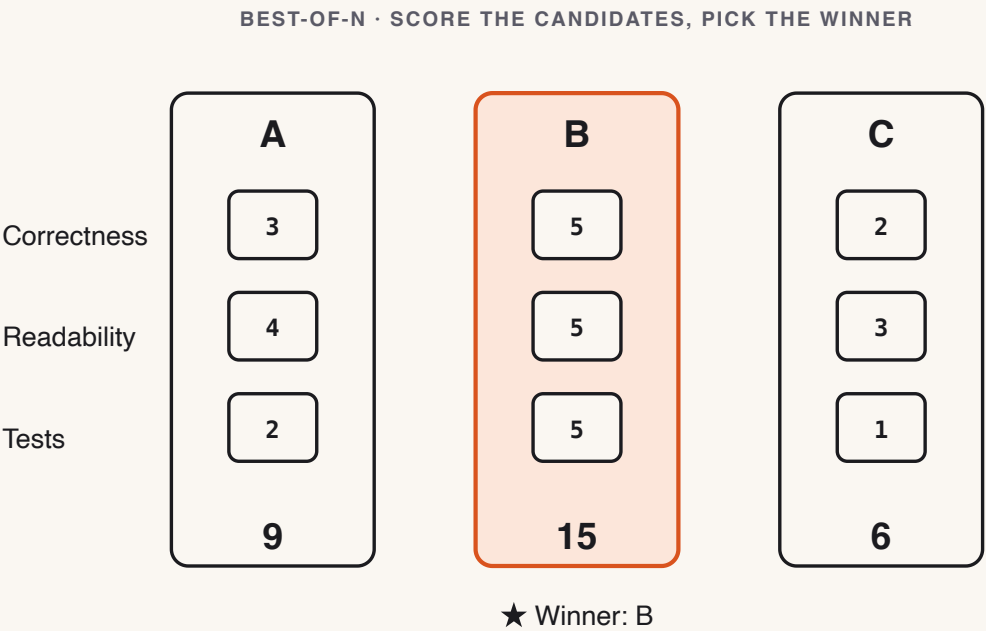
Theory · Best-of-N (4 min)

Generate N independent candidates → score on a rubric → pick the winner. N = 3 is the sweet spot.

- **Independent** — each candidate gets its own fresh prompt context. *Not* "now improve it" (that's iteration).
- **Score on a rubric**, not vibes. Three criteria:
 - **Correctness** — passes the manual test plan?
 - **Simplicity** — could a junior maintain it?
 - **Fit** — matches `CLAUDE.md` conventions and repo style?

Without the rubric you pick by gut and the lift disappears. Correctness gates everything.

Best-of-N, scored



Generate **N** independent candidates → score on **Correctness · Simplicity · Fit** → keep the winner.

Reference · The 3-criterion scorecard

Criterion	Question	Weight
Correctness	Does it pass every step of the manual test plan?	Gate — fail here = out
Simplicity	Could a junior maintain it next quarter?	High
Fit	Does it follow <code>CLAUDE.md</code> + repo style?	Medium

Record a one-paragraph justification per candidate in `scoring.md`. **Never delete losers before scoring.**

Reference · Common mistakes

- One candidate + "improve it" ×3 (iteration, not BoN).
- Skipping the rubric → picking by vibe → no lift.
- Choosing the "elegant" one that fails the test plan (correctness is the gate).

Live demo · Three candidates, one winner (6 min)

The reusable prompt — paste it verbatim for A, B, and C:

```
GOAL: A REST Notes API: create, list, get, delete a note.  
CONSTRAINTS: Python 3.11 + FastAPI; persist to SQLite ./notes.db;  
            return JSON; 404 on missing id; no other third-party deps.  
OUTPUT: one runnable app + a curl test plan covering all 4 routes.  
EXAMPLES: POST /notes {"text":"hi"} -> 201 + id;  
          GET /notes/999 (missing) -> 404 {"error":"not found"}.
```

`/clear` before **each** candidate → paste the **same** prompt → save to `candidate-a|b|c/`.
Then score side-by-side and commit the winner.

Variance must come from the model, **not** the prompt. Never say "now do it differently."

▶ Your turn · Notes API, Best-of-3 (13 min)

Exercise: [exercises/part-04/README.md](#)

Build a Notes API (SQLite `notes.db`), then generate **3 candidates** and score them:

```
POST /notes · GET /notes?q= · GET /notes/:id · PATCH /notes/:id · DELETE /notes/:id  
Status: 201 create · 200 read/update · 204 delete · 404 missing · 422 invalid
```

Track A: Python (FastAPI + Pydantic v2). Track B: Node (Hono + Zod + better-sqlite3).

Deliverables: `candidate-{a,b,c}/`, `scoring.md` (scores + justification), `winner/` (clean copy).

Success signal: all five endpoints respond with correct status codes via curl.

✓ Done & next (1 min)

Definition of done

- ☐ [] Three candidates in `module-04/candidate-{a,b,c}/`.
- ☐ [] `scoring.md` with scores + one-paragraph justification each.
- ☐ [] `winner/` runs end-to-end; losers archived (not deleted).

Next — we trust the winner only after we *test* it and review like a stranger's PR.

Module 5 — Testing, Debugging & Self-Review.